

Ripple: Report and Decision History

A self-improving harness that designs circuit boards from a spec

Marcos Speroni Jovian Arjun Cheema Jack Li

MongoDB Harness Engineering & Model Wrangling Hackathon
September 26, 2026

Abstract

Ripple turns an English spec such as “a board that reads a temperature sensor over I2C, powered from USB-C, with a status LED” into a routed printed circuit board, grades it against hidden checks the agents never see, and rewrites its own rules, context policy, tool access, workflow and model routing when the numbers say it should. Large boards are split into a work queue of subcircuits that are built, verified and assembled step by step, and the queue survives a kill and restart. MongoDB Atlas holds every spec, board, run, lesson, subcircuit and harness version. This document records why we built it, what we built, and every technical decision we made during the day, including the ones where we chose not to use a framework or tool that was on offer. The thesis of the project is one line: the model didn’t get smarter, the harness did.

Contents

1	Motivation	2
1.1	The problem the hackathon posed	2
1.2	Why PCB design	2
1.3	Why tscircuit	2
2	What we built	3
2.1	The design loop	3
2.2	The meta-harness	3
2.3	The long-horizon path	3
2.4	Atlas as the memory and state layer	4
3	Decision history	4
3.1	D1: Own TypeScript agent loop, not Strands, LangGraph or the Vercel AI SDK	4
3.2	D2: Our own transactions for checkpointing, not the LangGraph checkpointer	5
3.3	D3: Plain git, per-person branches and AGENTS.md rather than an IDE-driven workflow such as Kiro	5
3.4	D4: Hidden checks live in code, never in the database	5
3.5	D5: A parts whitelist, versioned in the config	6
3.6	D6: Pin tscircuit, and turn its supplier lookup off	6
3.7	D7: Voyage AI for embeddings and rerank, not Atlas Automated Embeddings	6
3.8	D8: OpenRouter with per-role model routing	6
3.9	D9: The gate has guardrails on the thing that rewrites guardrails	7
3.10	D10: Lessons are gated, reviewed and reversible	7
3.11	D11: Subcircuits connect only through named nets	7
3.12	D12: A passing run ends in manufacturable files	8
3.13	D13: Web app on Next.js and Vercel, read-only, polling in production	8
3.14	D14: Frozen contracts and a single batch runner	8
3.15	Decisions not to do things	8

4	Timeline	9
5	Evaluation	9
5.1	What is measured	9
5.2	Evidence so far	9
5.3	Ablation	10
5.4	Risks that materialised	10
6	Limitations and what we would do next	10
A	Repository map	10
B	Commands	11

1 Motivation

1.1 The problem the hackathon posed

The event asked for two things. **Recursive harnessing**: an agent system whose scaffolding, not just its prompts, is data that the system can inspect and improve. **Long-horizon engineering**: work that takes many steps, is checkpointed, and resumes after a crash without the context window growing without bound.

Most demos of “self-improving agents” improve a prompt. We wanted to improve the whole harness: which rules the coder is given, how much retrieved context it sees, which tools it may call and when, how many repair attempts it gets before escalating, and which model does which job. Each of those is a knob that a human harness engineer would tune by hand. Ripple keeps them in one versioned document and lets a meta-agent propose changes to it, with a gate that keeps, rolls back or rejects each one.

1.2 Why PCB design

We needed a domain with three properties.

1. **Checkable in code.** Whether a board has pull-ups on its I2C lines, a decoupling capacitor within a few millimetres of each supply pin, zero design-rule errors, and every required net connected is a deterministic function of the board’s netlist and layout. No LLM judge is needed, so the grade cannot be gamed by the thing being graded.
2. **Real first-attempt failures.** The mistakes behind most first-board respins are well known and small: missing pull-ups, missing decoupling, floating address pins, wrong LED polarity, a shared series resistor. A harness that learns those as rules is learning something a working engineer would recognise.
3. **Naturally long-horizon.** A 20-part board with USB-C power, a regulator, a microcontroller and two sensors is too much for one shot from a cheap model, but it decomposes cleanly into subcircuits that connect only through named nets. That gives the work queue something honest to do.

1.3 Why tscircuit

tscircuit expresses a board as TypeScript and React: `<resistor>`, `<chip>`, `<trace>` inside a `<board>`. That mattered for four reasons. The whole team could read and review the generated code without a CAD tool. The compiler, autorouter and design-rule checks run in Node, so compile, route and DRC are ordinary function calls in the loop. The output, Circuit JSON, is a plain array of elements that our own checker can walk. And the same Circuit JSON exports to Gerbers, a bill

of materials, a KiCad project and a 3D model, so a passing run ends in files a board house could accept, not a picture.

2 What we built

2.1 The design loop

1. Retrieve relevant lessons and verified subcircuits from Atlas (vector search, then rerank), sized by the harness config’s context policy.
2. The **coder** writes tscircuit source from the spec, the config’s rules, the retrieved context and the previous attempt’s diagnosis.
3. Compile to Circuit JSON, autoroute, run DRC.
4. The **hidden checker** grades the board: connectivity, I2C pull-ups, decoupling distance, tied pins, series LEDs, dividers and RC networks computed from part values, distinct I2C addresses, and zero DRC errors.
5. On failure the **critic** diagnoses the run, proposes the smallest fix, and writes lessons. The loop retries up to the config’s repair budget.
6. On success the subcircuit goes into the library, lessons that were in context are credited, and the deliverables are written.

2.2 The meta-harness

A batch of training specs runs under the current config. The **meta-agent** reads the batch’s failures, lessons and costs and proposes one change to the config with a rationale. The **config gate** first checks guardrails, then runs the candidate on a fresh batch and compares scores with its parent. The verdict is **kept**, **rolled_back** or **rejected**, and every version with its scores and verdict stays in `harness_versions`.

2.3 The long-horizon path

For a large spec the **planner** splits the board into subcircuits, each with its parts, headers and shared nets, validated in code (whitelisted parts, every header owned by exactly one subcircuit, no cycles). Items go into a durable **work queue** in Atlas. Each step runs the coder for one subcircuit, checks its interface against the plan, and commits the run, the board document and the queue status in one transaction. The **assembler** reads the finished groups back, packs them without overlap, sizes the board, evaluates the combined source and grades it. Killing the worker mid-step and restarting it resumes from the queue with nothing lost.

2.4 Atlas as the memory and state layer

Collection	One document is	Used for
<code>harness_versions</code>	a harness config	rules, context policy, tool access, workflow, routing, parent, scores, verdict, rationale
<code>specs</code>	a board task	spec text and train or held-out split; never the checks
<code>subcircuits</code>	a verified building block	code, description embedding, checks passed, reuse count
<code>boards</code>	a board version	code, Circuit JSON, metrics, harness version used
<code>work_queue</code>	a step of a large board	status, dependencies, heartbeat; drives kill-and-resume
<code>runs</code>	one attempt or tool result	structured result, cost, model; the replayable trace
<code>lessons</code>	a learned design rule	pattern, fix, embedding, how often it helped, active flag
<code>spec_requests</code>	a request from the web app	inserted by a restricted user, claimed by the worker

Atlas features in use: Vector Search on three collections, versioned documents for the config history, multi-document transactions and idempotent upserts for the queue, and a change stream that lets the worker pick up spec requests from the web app.

3 Decision history

Each decision below gives the context, the options we had, what we chose, why, and what happened afterwards. The order is roughly chronological.

3.1 D1: Own TypeScript agent loop, not Strands, LangGraph or the Vercel AI SDK

Context. The design doc listed three candidate agent frameworks: the Strands SDK for the inner agents, LangGraph.js for the graph and its checkpointing, and the Vercel AI SDK for model calls. The default written before kickoff was “own TypeScript loop, unless someone has built with Strands before”.

Decision. A plain TypeScript loop with no agent framework. Model calls go through one small router; agents are functions that take their dependencies (the model call, the lesson store) as arguments.

Reasons.

- The harness config is the product. Every knob the meta-agent can turn (retrieval k , rerank on or off, which tools an agent may use, the repair budget, plan-first, model per role) has to be a field in a document that the gate can guard and the UI can diff. Frameworks bake most of those choices into graph structure or class configuration, which would have meant a second, implicit harness beside the one we were showing.
- Nobody on the team had shipped with Strands. Learning it on the morning of a one-day event was a risk with no upside for the demo.
- Strands’ own Harness Optimizer tunes an agent’s context (system prompt, tool docs, skills) from rollouts and rewards. That is a subset of what Ripple evolves, so the honest relationship is as a baseline, not a foundation. It stayed on the nice-to-have list as an ablation row and was not reached.

- Dependency injection made every agent testable with a fake model. The critic, planner and assembler each have a test suite that runs without a network.

Outcome. The router is 80 lines. Swapping the critic’s model from GPT-4o-mini to Gemini Flash was an environment variable. The whole loop, gate and ablation runner call the same `runBatch`.

3.2 D2: Our own transactions for checkpointing, not the LangGraph checkpoint

Context. Long-horizon work needs durable steps. LangGraph ships a checkpoint; the alternative was to write the queue directly on Atlas.

Decision. Own queue on Atlas, decided at the 11:00 go/no-go. A step’s run record, its board update and its queue status commit in one multi-document transaction. Run ids are deterministic, so a retried step overwrites rather than duplicates. Workers heartbeat every 30 seconds and stale claims are recovered.

Reasons. A checkpoint stores an opaque graph state. We wanted every step to be a first-class document that the UI can show, the gate can score and a teammate can query, and we wanted the durability story to be Atlas features rather than a library’s serialisation format.

Outcome. The finale board was killed with `kill -9` while the sensors subcircuit was running; the rerun recovered the item, finished, assembled and passed the hidden checks. Atlas afterwards showed each step exactly once.

3.3 D3: Plain git, per-person branches and AGENTS.md rather than an IDE-driven workflow such as Kiro

Context. Spec-driven agentic IDEs were available and would have given us a shared task and spec system inside the editor.

Decision. Keep the repository tooling-neutral: one branch per person, pull requests into `main`, and an `AGENTS.md` that states ownership, frozen contracts, the hidden-check boundary, the budget rules and the pinned versions. Each person used whichever coding agent they already had.

Reasons. Ripple already keeps its specs, rules and history in Atlas; an IDE-level spec system would have been a second source of truth for the same things, and one that judges could not see. With four laptops and one day, the cost of aligning everyone on a new environment outweighed the benefit. `AGENTS.md` did the job that an IDE workflow would have: it told both humans and coding agents where they may edit, what may not change, and what must stay hidden.

Outcome. Well-defined pieces with testable contracts were delegated to a coding agent and reviewed through pull requests like any other contribution: the hidden checker engine and its broken-board tests, the parts whitelist, the reference boards for all twelve specs, and the assembler. Where two people built the same thing in parallel, the version with the better design won and the other was closed.

3.4 D4: Hidden checks live in code, never in the database

Decision. All check definitions live under `checks/`. Nothing in that folder may be imported by an agent, put into a prompt, embedded for retrieval or exposed through MCP. The database holds only spec text and the train or held-out label.

Reasons. If the checks were data, the meta-agent could read them, and a “self-improving” harness that reads its own exam is not improving. Keeping them out of Atlas also keeps them away from the read-only database users the web app and MCP server use.

Design. Check files refer to parts by role and header label, never by component name, because the coder chooses its own names. `header:SDA` means every pin-header pin labelled SDA; `role:temp_sensor.VCC` means the canonical supply pin of any whitelisted temperature sensor. Rules are composed from shared blocks (USB-C sink, LDO with input and output capacitors, I2C bus with pull-ups, MCU basics). Analog rules (divider ratio, RC time constant and cutoff) are computed from part values rather than simulated.

Proof. Every spec has a reference board built only from whitelisted parts, and a test renders each one and asserts it passes its own checks with zero DRC errors. A spec without a passing reference is not solvable, and a checker with no passing case is not trustworthy.

3.5 D5: A parts whitelist, versioned in the config

Decision. The coder may use only parts from `parts/whitelist.json`: 25 entries covering passives, LEDs, a diode, headers, a USB-C receptacle, two 3.3 V regulators, an ATtiny85 and an STM32F030, an LM75, an SHT40, a BME280 and a 24LC256 EEPROM. Each entry carries the exact footprint string, pin labels, a manufacturer part number, roles and canonical pins. The whitelist version is a field of the harness config, so narrowing it is a change the meta-agent can propose and the gate can refuse.

Reasons. An agent that invents footprints produces boards that never render. A script renders every whitelisted part through the pinned `tscircuit` and fails on any error; QFN-32 and DFN-8 were dropped because they fail pad-clearance DRC at that version.

3.6 D6: Pin `tscircuit`, and turn its supplier lookup off

Decision. `tscircuit@0.0.2646` exactly, never upgraded, never `npm audit fix`. The evaluator runs with `partsEngineDisabled`.

Reasons. The autorouter and footprint library change between releases; a mid-day upgrade would have invalidated the whitelist and the reference boards. The supplier lookup was found the hard way: the evaluator attached a JLCPCB part to every LED and then reported a polarity mismatch that the direct render path never raised, which failed the first checkpoint attempt at 12:19. Turning the lookup off fixed it and also made the loop work offline.

3.7 D7: Voyage AI for embeddings and rerank, not Atlas Automated Embeddings

Decision. Voyage `voyage-4` embeddings (1024 dimensions) and `rerank-2.5`, called from the worker; three vector indexes, on `lessons`, `subcircuits` and `runs`, which is the limit on an M0 cluster.

Reasons. The design doc gave Automated Embeddings twenty minutes of trying and Voyage as the default. Calling Voyage directly gave us control over what is embedded: short descriptions and lesson text, never raw code, so a lesson retrieves on meaning. The reranker is a config knob, so the meta-agent can turn it off to save cost.

Outcome. Lesson deduplication uses the same vectors: a new lesson whose nearest existing lesson scores at or above 0.965 updates that lesson instead of adding a near-duplicate. The threshold was verified against the live lessons written during the first evolve run.

3.8 D8: OpenRouter with per-role model routing

Decision. One router maps an agent role to a tier through the harness config and a tier to a model id. Defaults are `openai/gpt-4o-mini` for cheap and `anthropic/claude-sonnet-4.5` for strong,

overridable by environment variables. Cost comes back from OpenRouter’s usage field rather than a price table. Every call is traced to LangSmith.

Reasons. Model routing is one of the knobs the meta-agent controls, so it had to be data. Real cost per call is what the gate compares. The budget rules in `AGENTS.md` became stricter during the day: each person got their own small-budget key, cheap models by default, and `max_tokens` always set.

Outcome. In practice the critic and planner ran on Gemini Flash, at \$0.006 to \$0.013 per call. The first evolve round is the best evidence for routing as data: the meta-agent proposed moving the coder to the strong model, and the gate rolled it back because the pass rate stayed at zero while cost per board rose from \$0.002 to \$0.078.

3.9 D9: The gate has guardrails on the thing that rewrites guardrails

Decision. Before any batch runs, a proposed config is rejected if it removes a rule, turns off connectivity-before-route, swaps the parts whitelist, gives the coder any MCP tools, grants non-read-only MCP tools, sets the repair budget outside 1 to 6, splits boards below 4 parts, or sets retrieval k outside 0 to 10. Otherwise the candidate runs on a fresh batch and is kept only if its scores beat the parent’s.

Reasons. A meta-agent optimising a score will eventually try to weaken the check that produces it. The rejection path had to exist and had to be visible, so rejected versions are stored with their reason next to kept and rolled-back ones.

3.10 D10: Lessons are gated, reviewed and reversible

Decision. The critic’s lessons pass a quality gate in code: a lesson that names a component designator, a spec id, or a routing constraint is rejected, and held-out specs never produce lessons so the ablation stays clean. Lessons can be retired and restored with a script; retirement keeps the reason.

Reasons and outcome. The first review of the twelve lessons from the evolve run retired three. One said “decoupling cap within 1 mm” and claimed that autorouters enforce a 1 mm trace limit; the check requires 3 mm, the coder turned the lesson into a trace-length limit, and autorouting was skipped. Two were duplicates. The critic prompt was tightened in response: use the failure’s own numbers, never invent a stricter limit, never add trace constraints, and fix placement before wiring when autorouting was skipped for courtyard errors. This is the loop working as intended: the harness learned something wrong, the gate and a human caught it, and the correction became part of the harness.

3.11 D11: Subcircuits connect only through named nets

Decision. A subcircuit is a tscircuit component that returns a `<group name=...>` and talks to the rest of the board only through `net.V3V3`, `net.GND`, `net.SDA` and so on. Component names are unique across the board. The assembler measures each group alone, shelf-packs the bounding boxes, sizes the board, writes one source file and evaluates it.

Reasons. A queue only helps if items are independent. Net names are the one interface that the planner can hand every coder in advance and that the interface check can verify per step. Packing by measured bounding boxes means groups never overlap by construction; the autorouter then routes across them.

Outcome. The finale’s three groups (power, MCU with ISP header, sensors) assemble into a 47 by 38 mm board with 20 parts and zero DRC errors, deterministically.

3.12 D12: A passing run ends in manufacturable files

Decision. Every finished board exports Gerber layers and drill files, a BOM with manufacturer part numbers, pick-and-place, an assembly drawing, a KiCad project, a 3D model, PCB and schematic images, a netlist, a SPICE netlist, a design report and a manifest for the front end. Exporters are pinned exactly.

Reasons. The stated non-goal is proving a board works on real hardware; the checks prove spec and rule compliance. Shipping the files is the honest way to show what the harness produced and lets anyone check it in a real tool.

3.13 D13: Web app on Next.js and Vercel, read-only, polling in production

Decision. The UI is a Next.js app scaffolded with v0 and deployed on Vercel. It reads Atlas through a read-only user. Locally it can stream change streams; on Vercel it polls, because a serverless function cannot hold a stream open. A judge's spec goes in through an insert-only user into `spec_requests`; the worker claims it with a change stream.

Reasons. One writer. The worker is the only process that writes design state, so a UI bug or a leaked reader URI cannot corrupt a run. The 3D board view was chosen over a schematic-first view because a routed PCB changing in front of the audience is the demo.

3.14 D14: Frozen contracts and a single batch runner

Decision. `HarnessConfig` and `RunResult` were frozen at kickoff and change only with the whole team's agreement. `runBatch` is the only batch runner; the gate, the meta-agent and the ablation all call it, and the config is caller-supplied so any version can be run.

Reasons. Four people integrating in one afternoon need stable edges. Freezing the two documents that every component reads or writes let the checker, the queue, the router and the UI be built in parallel and meet at 12:30.

3.15 Decisions not to do things

- **No SPICE in the checks.** Analog specs (a 12 V divider, an RC low-pass, a debounced button) are graded by computing the ratio or time constant from part values. It is exact for the circuits in scope and avoided a simulator dependency in the loop.
- **No Strands Harness Optimizer baseline and no cheap-versus-frontier ablation row.** Both were nice-to-haves behind the finale board and were not reached.
- **No model training or fine-tuning.** The thesis is that the harness improves; the model is fixed.
- **Read-only MCP access last.** The critic and meta-agent can work from the batch summary the loop hands them; direct database access was intentionally the final item.
- **Boards stay small.** Two layers, roughly 20 parts. The autorouter is the slowest and least predictable component, and every risk we listed about it was real.

4 Timeline

Time	What landed on <code>main</code>
10:39	Design doc, repo layout, ownership, pinned tscircuit, LED smoke board
10:51	Workspace scaffold: pinned deps, frozen contracts, index setup, green check
11:03	8 training and 4 held-out specs with hidden requirements
11:07	Hidden checker (connectivity, pull-ups, decoupling, DRC), parts whitelist v1
11:27	Versioned harness config; v0 seeded
11:33	Checker with role and header references and rules for all 12 specs; whitelist v2
11:41	Memory: Voyage embeddings, vector search, rerank; 3D front end
11:47	Reference boards for all 12 specs with a pass test
11:58	Finale spec, checks and 20-part reference in three groups
12:02	Compile, evaluate, DRC, metrics, router, coder
12:03	Durable work queue with kill-and-resume
12:05	Critic agent with lesson quality gate
12:08	Assembler
12:15	Planner agent
12:19	First checkpoint attempt fails on the JLCPCB LED polarity error
12:33	Parts engine off; checkpoint passes on <code>t01</code> with the cheap model
12:36	Deliverables export
12:40	Config gate: scores, guardrails, verdicts
12:41	Critic in the loop, <code>runBoard</code> and <code>runBatch</code> , meta-agent, ablation runner
12:45	Finale queue handler; kill-and-resume dry run passes hidden checks
13:05	<code>npm run evolve</code> : first round rolls back “coder to strong”
13:07	Front end reads Atlas; single-board Circuit JSON; honest ablation cost
13:25	Lesson review: 3 of 12 retired; critic prompt and gate tightened
13:31	Spec requests worker with change stream; lesson dedupe at 0.965

5 Evaluation

5.1 What is measured

Metric	Source
Hidden checks passed per board	checker
Attempts per board	<code>runs</code>
Cost per board	OpenRouter usage per call, summed in <code>runs</code>
Board area, vias, trace length	Circuit JSON
Config changes kept, rolled back, rejected	<code>harness_versions</code>

5.2 Evidence so far

- **Checkpoint.** One training spec goes from text to a routed board that passes its hidden checks on the first attempt with the cheap model, under a stored config version.
- **Gate.** The first evolve round on two training specs: v0 passed 0 of 2; the meta-agent proposed routing the coder to the strong model; the candidate also passed 0 of 2 at 37 times the cost per board, and the gate rolled it back. The critic wrote 12 lessons during the round.
- **Lessons.** After review, 9 active lessons remain, covering I2C pull-ups, address pins, write-protect, supply and capacitor wiring, decoupling distance and courtyard-aware placement.
- **Long horizon.** The finale board built through the queue, was killed mid-step, resumed, assembled, passed every hidden check, and produced 30 deliverable files.

5.3 Ablation

The ablation runs the same model on the four held-out specs under three conditions, with memory writes off so held-out solutions never enter the library. The table is produced by `npm run ablation` and is to be filled from that run.

Setup	Checks passed	Attempts per board	Cost per board
Bare model, no harness			
Harness v0: loop and checks, no memory			
Harness vN: evolved config, library, lessons			

5.4 Risks that materialised

- The evaluator’s supplier lookup rewrote LED polarity (D6).
- A learned lesson was wrong and harmful until reviewed (D10).
- Windows checkouts with CRLF line endings broke the finale reference parser and the assembler; fixed with `.gitattributes`.
- The pinned autorouter occasionally crashes on a re-render in one process; tests pin only the deterministic netlist rules to exact results.
- The meta-agent’s batch summary counted failures by check name only, so it could not see that courtyard overlaps were the cause of skipped autorouting; the fix is to give it the top failure details and the critic’s lessons.

6 Limitations and what we would do next

Passing the checks proves spec and rule compliance, not that a board works; nothing here replaces a review by an engineer before ordering. The domain is deliberately narrow: two layers, about 20 parts, no high-speed or signal-integrity concerns. The meta-agent so far proposes one change per round and has been better at proposing model changes than rules; feeding it failure details and lessons is the obvious next step. The Strands Harness Optimizer would be a fair external baseline for the context-tuning part of what Ripple does. And the ablation, the single most persuasive artifact, needs its numbers.

A Repository map

Path	Contents
<code>apps/worker/src/agents/</code>	coder, critic, planner, meta-agent
<code>apps/worker/src/tools/</code>	compile, evaluate, DRC, metrics, router, parts whitelist
<code>apps/worker/src/harness/</code>	config versions, memory, queue, gate
<code>apps/worker/src/assembler/</code>	subcircuit groups to one board
<code>apps/worker/src/pipeline/</code>	planned board through the queue
<code>apps/worker/src/export/</code>	deliverables
<code>apps/web/</code>	Next.js UI
<code>checks/</code>	hidden checker, per-spec checks, reference boards (agents never read)
<code>specs/</code>	public spec text
<code>parts/</code>	whitelist
<code>scripts/</code>	green check, index setup, evolve, ablation, finale, lessons
<code>packages/types/</code>	frozen contracts

B Commands

Command	Does
<code>npm run green</code>	checks Atlas writer and reader, Voyage, one traced model call
<code>npm run smoke [spec]</code>	renders a reference board through autorouter and DRC
<code>npm run test:checks</code>	checker unit tests, broken variants, all reference boards
<code>npm run test:planner,</code> <code>test:assembler</code>	agent and assembler tests with fake models
<code>npm run evolve - -rounds</code> <code>N</code>	batch, meta-agent proposal, gate verdict
<code>npm run ablation</code>	bare versus v0 versus evolved on held-out specs
<code>npm run finale [- -stub]</code>	the long-horizon board through the queue
<code>npm run lessons</code>	list, retire, restore lessons
<code>npm run worker</code>	serve spec requests from the web app